RPL Homework 3
Reinforcement Learning of Robot Policies

Robot Perception and Learning
NTU CSIE, Fall 2025

Outline

- Introduction: Policy Optimizations
- Problems: GridWorld
- Code Implementation
- Grading
- Package
- Submission

Policy Proximal Optimization (PPO)

- Policy gradient objective:

$$\max_{\theta} \; \mathbb{E}_{a_t \sim \pi_\theta(a_t|o_t)} \left[\log \pi_\theta(a_t \mid o_t) \, \hat{A}_t \right],$$

- But large updates can lead to policy collapse → incorporates a trust region by clipping the ratio:

$$\max_{\theta} \; \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(a_t|o_t)} \left[\min\left(r(\theta)\hat{A}_t, \boxed{\text{clip}\left(r(\theta), 1-\varepsilon^{\text{clip}}, 1+\varepsilon^{\text{clip}}\right)} \hat{A}_t \right) \right],$$

  - Ratio between current and old likelihoods:

$$r(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{old}(a_t \mid o_t)}$$

[1] Schulman, John, et al. "Proximal policy optimization algorithms." arXiv preprint arXiv:1707.06347 (2017).

Trust Region Policy Optmization (TRPO)

- PPO can still suffer from occasional performance collapse when updates are too large or the policy distribution shifts too quickly, TRPO[1] introduces an explicit KL-divergence constraint (a trust region) to ensure each policy update stays within a safe range and reliably improves performance.

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_{s,a\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A_{\pi_{\text{old}}}(s,a)\right] \\ \text{subject to} \quad & \mathbb{E}_{s\sim d_{\pi_{\text{old}}}}\left[\mathrm{KL}(\pi_{\text{old}}(\cdot \mid s) \parallel \pi_\theta(\cdot \mid s))\right] \;\leq\; \delta \end{aligned}$$

- Further details can be found in RL — Trust Region Policy Optimization (TRPO) Explained

[2] Schulman, John, et al. "Trust region policy optimization." *International conference on machine learning*. PMLR, 2015.

From PPO to Flow Policy Optimization (FPO)

- In PPO, it requires exact likelihoods for sampled actions → computationally prohibitive
- FPO[3] propose a proxy for the log-likelihood ratio:

$$\max_{\theta} \; \mathbb{E}_{a_t \sim \pi_\theta(a_t|o_t)} \left[\min\left(\hat{r}^{\mathrm{FPO}}(\theta) \hat{A}_t, \; \mathrm{clip}\left(\hat{r}^{\mathrm{FPO}}(\theta), 1 - \varepsilon^{\mathrm{clip}}, 1 + \varepsilon^{\mathrm{clip}} \right) \hat{A}_t \right) \right].$$

- A simple ratio estimate using standard flow matching loss:

$$\hat{r}^{\mathrm{FPO}}(\theta) = \exp\left(\hat{\mathcal{L}}_{\mathrm{CFM},\theta_{\mathrm{old}}}(a_t; o_t) - \hat{\mathcal{L}}_{\mathrm{CFM},\theta}(a_t; o_t) \right),$$

[3] McAllister, David, et al. "Flow matching policy gradients." *arXiv preprint arXiv:2507.21053* (2025).

Theoretical Foundation: FPO Surrogate Objective

- Exact likelihood is computationally expensive even to estimate in flow-based models. Instead, it is common to optimize the evidence lower bound (ELBO)[4] as a proxy for log-likelihood:

$$\mathrm{ELBO}_\theta(a_t \mid o_t) = \log \pi_\theta(a_t \mid o_t) - \mathcal{D}_\theta^{\mathrm{KL}}$$

- we compute the ratio of ELBOs under the current and old policies:

$$r(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{old}(a_t \mid o_t)} \quad \Rightarrow \quad r^{\mathrm{FPO}}(\theta) = \frac{\exp\big(\mathrm{ELBO}_\theta(a_t \mid o_t)\big)}{\exp\big(\mathrm{ELBO}_{\theta_{\mathrm{old}}}(a_t \mid o_t)\big)}.$$

[4] Kingma, Diederik, and Ruiqi Gao. "Understanding diffusion objectives as the elbo with simple data augmentation." *Advances in Neural Information Processing Systems* 36 (2023): 65484-65516.

Theoretical Foundation: Estimating the FPO Ratio

- For the diffusion schedule, [4] proves that the weighted denoising loss is equivalent to the ELBO:

$$\mathcal{L}_\theta^w(a_t) = -\mathrm{ELBO}_\theta(a_t) + c,$$

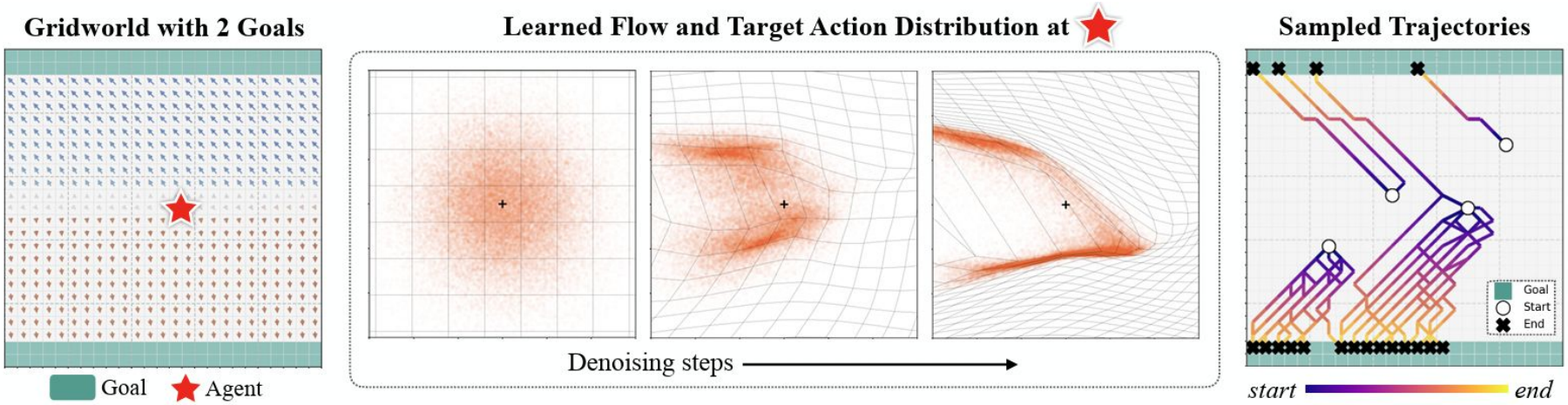
- That brings us back to the approximation in FPO:

$$r_\theta^{\mathrm{FPO}} = \frac{\exp\big(\mathrm{ELBO}_\theta(a_t \mid o_t)\big)}{\exp\big(\mathrm{ELBO}_{\theta_{\mathrm{old}}}(a_t \mid o_t)\big)} = \exp\big(\mathcal{L}_{\theta_{\mathrm{old}}}^w(a_t) - \mathcal{L}_\theta^w(a_t)\big),$$

[4] Kingma, Diederik, and Ruiqi Gao. "Understanding diffusion objectives as the elbo with simple data augmentation." *Advances in Neural Information Processing Systems* 36 (2023): 65484-65516.

Environment – Toy 25x25 GridWorld

- An agent in a 25×25 grid must reach one of two high-reward goal regions located at the top or bottom of the map, with only a single sparse reward (or penalty) upon reaching a goal.



Code Implementation Part 1: PPO (Warm-up)

- Please complete the TODO in the learn() function of the PPO class in models/ppo.py, which mainly concerns implementing the clipped surrogate objective:

$$\max_{\theta} \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(a_t|o_t)} \left[\min\left(r(\theta)\hat{A}_t,\ \text{clip}\left(r(\theta), 1-\varepsilon^{\text{clip}}, 1+\varepsilon^{\text{clip}}\right)\hat{A}_t \right) \right],$$

- Steps
  - Compute the probability ratio r_t(θ)
  - Compute the clipped surrogate objective
  - Add entropy bonus (to encourage exploration)
  - Compute value function loss (critic_loss) using MSE
  - Compute approximate KL divergence for early stopping

Code Implementation Part 2: TRPO (Trust-Region Step Scaling)

- Please complete the TODO 1 in the learn() function of the TRPO class in models/trpo.py, which mainly concerns implementing trust-region step scaling:
- Goal, compute the update: $\Delta\theta = -\text{step_size} \cdot s, \qquad s \approx F^{-1}g$
- TRPO requires the step to satisfy: $\frac{1}{2}s^\top F s \leq \text{max_kl}$
- To compute the step size (TODO):

$$Fs \approx \text{Avp_func}(s)$$

$$s^\top F s = \sum_i s_i (Fs)_i$$

$$\text{step_size} = \sqrt{\frac{2\,\text{max_kl}}{s^\top F s + 10^{-8}}}$$

Code Implementation Part 2: TRPO (Line Search Acceptance Rule)

- Please complete the TODO 2 in the learn() function of the TRPO class in models/trpo.py, which mainly concerns implementing line search acceptance rule:
- For each backtracking coefficient: $\theta_{\text{new}} = \theta_{\text{old}} + \alpha^j \Delta\theta$
- A candidate update is accepted only if both conditions are met
  - Surrogate loss improves: $L_{\text{new}} \leq L_{\text{old}}$
  - KL remains within the trust region: $\mathrm{KL}(\pi_{\text{old}} \parallel \pi_{\theta_{\text{new}}}) \leq \text{max_kl}$
- If both hold → accept the update; otherwise → continue backtracking.

Code Implementation Part 2: TRPO (Surrogate Objective)

- Please complete the TODO 3 in the learn() function of the TRPO class in models/trpo.py, which mainly concerns implementing Surrogate Objective:

$$\max_{\theta} \quad \mathbb{E}_{s,a\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A_{\pi_{\text{old}}}(s,a)\right]$$
$$\text{subject to} \quad \mathbb{E}_{s\sim d_{\pi_{\text{old}}}}\left[\mathrm{KL}(\pi_{\text{old}}(\cdot \mid s) \parallel \pi_\theta(\cdot \mid s))\right] \leq \delta$$

- Steps
  - Build current policy distribution
  - Compute log_probs under current policy
  - Compute ratios r_t(θ)
  - Surrogate loss (note the negative sign, since we MINIMIZE)
  - Add entropy bonus
  - KL divergence (for trust region)

Code Implementation Part 3: FPO

- Please complete the TODOs in the learn() function of the FPO class in models/fpo.py, which mainly concerns implementing the CFM loss difference (part 1), the state-wise policy ratio ρ□ (part 2), and the PPO-style clipped surrogate objective (part 3):

$$\max_{\theta} \ \mathbb{E}_{a_t \sim \pi_\theta(a_t|o_t)} \left[\min\Big(\hat{r}^{\mathrm{FPO}}(\theta)\hat{A}_t, \ \mathrm{clip}\big(\hat{r}^{\mathrm{FPO}}(\theta), 1-\varepsilon^{\mathrm{clip}}, 1+\varepsilon^{\mathrm{clip}}\big)\hat{A}_t \Big) \right].$$

$$\hat{r}^{\mathrm{FPO}}(\theta) = \exp\big(\hat{\mathcal{L}}_{\mathrm{CFM},\theta_{\mathrm{old}}}(a_t; o_t) - \hat{\mathcal{L}}_{\mathrm{CFM},\theta}(a_t; o_t)\big),$$

Grading

- (50%) The implementation of 3 policy optimization (PPO, TRPO, FPO) on GridWorld
- (40%) Report (in your README.md)

  a. For all three policy optimization methods, each evaluation rollout is capable of reaching the goal region (i.e., achieving a return of 20). You may justify this in any valid manner.
  b. Using the provided Weights & Biases tools, plot the average_actor_loss curves for PPO, TRPO, and FPO during training. Compare their differences and describe what do you observe?
  c. Using the provided eval_and_visualize_trajectories.py to visualize multiple evaluation runs for PPO, TRPO, and FPO. Compare their differences and describe what do you observe?

- (10%) Challenge – Evaluate PPO, FPO on another *multimodal* environment (e.g., MuJoCo Playground, puffer-phc) and do comparison and analyze whether its diffusion-based policy better captures multimodal actions.

Packages

- Please follow the README to install the required packages.
- If you need to install any additional packages, please ask the TA in the Discord channel and provide usage instructions in the README.md.

Submission

- Deadline: 2025/12/14 (Sun.) 23:59 (GMT+8)
- Please click the link below and sign in to your GitHub account to access your submission repository: https://classroom.github.com/a/C4kPc08K
- Kindly link your GitHub account to the classroom with your student ID.
- If your student ID is not listed, please contact TA by Discord.
- We will clone the main branch of your repository.
- Please submission your GitHub classroom link to NTU COOL Homework 3.
- The score of the assignment will be multiplied by 0.9 for each additional day of delay. No grace day is granted.

Final Reminder

- Do NOT send private messages to TAs via Facebook or spam TAs email.
- TAs will NOT debug for you (e.g., coding, environment, dependencies, etc. issues).
- As mentioned in the first class, we DO NOT provide any computation resources.
- TA is here to help. Discussions and Q&As are welcome. Question can be asked in Discord hw3 discussion group: https://discord.com/channels/1410981038825410590/1411400018933579846